

G07 / PROTECTED ANSWERS

G07 REVIEW A

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

A concrete project defense and STAR example 5

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Final stage boundary | Master untimed first; optional interview timing only after a correct cold attempt

Protected review: use only after a saved Gate attempt. This is a worked repair guide, not a script to memorize. Repair weak domains, close this file, then take the different fresh gate.

1 | Truth audit - acceptable repair

Hypothetical overclaim: "I am a Data Engineer with 3 years of production experience building enterprise-scale Spark pipelines for millions of users." If the evidence is course/portfolio work, rewrite to: "Data-engineering portfolio focused on replay-safe batch pipelines, analytics engineering, Spark/lakehouse processing and production reliability; projects include saved validation and failure-recovery evidence." Keep only claims your repositories or real employment can prove.

2 | SQL - solved shape

Output grain is one row per hub for delivered amount, plus a separate scalar failed-shipment count. One acceptable SQLite solution:

```
SELECT hub, COUNT(*) AS delivered_shipments,  
       ROUND(SUM(amount), 2) AS delivered_amount  
FROM shipments  
WHERE status='DELIVERED'  
GROUP BY hub  
ORDER BY hub;
```

```
SELECT COUNT(*) AS failed_shipments  
FROM shipments  
WHERE status='FAILED';
```

- Expected: H1 delivered=120.00; H2 delivered=250.00; delivered total=370.00; delivered shipments=3; failed
- shipments=1.
- Reconcile 4 source rows = 3 delivered + 1 failed. A correct number without stated grain/reconciliation is incomplete.

3 | Python - solved shape

Use one set for duplicates and one frequency map for status counts. For the supplied shipment IDs there are no duplicates; counts are DELIVERED=3 and FAILED=1. A one-pass solution is $O(n)$ time and $O(n)$ space in the worst case.

```
seen = set()  
dup = set()  
counts = {}  
for r in rows:  
    sid = r["shipment_id"]  
    if sid in seen:  
        dup.add(sid)
```

```
else:
    seen.add(sid)
counts[r["status"]] = counts.get(r["status"], 0) + 1
```

4 | Troubleshooting - safe checkpoint recovery

The output may already be durable even though the checkpoint was not committed. Freeze new progress; inspect durable sink identity/uniqueness and the last committed checkpoint; determine exactly which records are safe to replay; rerun idempotently; then reconcile source IDs/counts before advancing progress. Do not blindly delete output or advance the checkpoint to “catch up.”

5 | Architecture - one defensible design

For 2M shipments/day with daily reporting plus 15-minute exception analytics: immutable raw landing -> validated/quarantined trusted layer -> 15-minute micro-batch exception mart + daily aggregate mart -> BI/alerts. Keep explicit run/checkpoint state, dedupe/replay identity, count/amount reconciliation, freshness/lag alerts and a bounded backfill route. Streaming is optional only if the 15-minute SLA cannot be met simply. Discuss PII/RBAC and compute/storage cost.

6 | Project defense - pass shape

A pass answer states business need, row/event grain, architecture, one hard trade-off, one reproduced failure/recovery, independent validation and one honest limitation. “I used Spark, Kafka and Airflow” without those

boundaries is not a defense.

7 | Behavioral + technical English - pass shape

STAR may use a real course-project failure if labelled truthfully. Situation/Task are short; Action explains reproduction, evidence and repair; Result states the verified outcome. Manager version removes implementation detail but keeps the same facts, impact and prevention.

8 | Job targeting - verdict

APPLY/REVIEW. “Bachelor degree or equivalent practical experience” explicitly allows a non-degree-equivalent evidence route; SQL/Python are required and must be proven; Spark is preferred rather than a mandatory gate. Do not convert “equivalent” into a guaranteed interview - apply only if the rest of the requirements fit.

Common failure modes

- Fake employment/scale claims.
- SQL totals with no stated grain or reconciliation.
- Checkpoint recovery that deletes durable output or advances over failed work.
- Architecture that lists tools but omits replay/state, failure, security or cost.
- Behavioral answer that says “we” without explaining your actual action.

Repair rubric

- Conceptual prose cannot replace requested runnable SQL/Python evidence.
- Correct numbers without grain and independent controls are partial.
- Architecture without state/replay, quality, failure, security and cost is partial.
- Communication may be simple English; technical truth and structure matter more than accent.
- After repair, close this review and take the different fresh gate without reading these worked solutions.

A concrete project defense and STAR example

Illustrative practice answer — use it only after reproducing the scenario, and replace it with your own verified experience. “In my local P01 portfolio, the business question was paid order revenue. Day one had nine accepted current orders and paid revenue 2150. A later three-change batch gave eleven current orders and paid revenue 2940. I interrupted publication before the current-result pointer advanced. I checked that the earlier generation stayed visible, then replayed the same input and reconciled the new generation. I used immutable input and a controlled publication boundary to avoid a half-written result. This reference is single-writer and local; I have not demonstrated distributed transactions.”

STAR version: Situation — my local course pipeline could be interrupted during publication. Task — keep readers on a complete result and make replay safe. Action — preserve raw input, stage a new result, inject the failure before publication, inspect the visible generation and replay. Result — only if your own run proves it, report that the old generation remained readable and the rerun produced the expected eleven rows and 2940. Do not turn this into employment or large-scale production experience.

Manager version: “An interrupted update could expose incomplete numbers. I tested an interruption, kept the previous complete report available and made the update repeatable. My checks compare the order count and paid total before I call the update complete.”

For B choose another project and real failure, for example P02's duplicate invoice: explain why aggregation before checking invoice identity could overstate revenue, show the blocked duplicate fixture, then reproduce the valid 1240 total. At 20× volume inspect scan time, partition sizes and join/shuffle evidence before selecting a change. A larger cluster cannot repair a duplicate-invoice contract.